

eBay Valuation Bot: the model, the learning loop, and go-live

How it prices, where it learns, what you can tune · 17 June 2026 · read-only (it never bids)

Short version: there's no black-box AI doing the pricing. It's a transparent statistics model you can read top to bottom, with every dial exposed in one config file. The "learning" is a deliberate human-in-the-loop calibration loop rather than a self-training model: it tells you what to nudge, you decide. Here's the whole thing on a page.

How it prices one item



More than an average: an estimate that resists noise. It takes the **median** of comparable prices, not the mean, so one freak sale can't drag the number. Then it multiplies by a **sold-to-asking ratio** and subtracts $k \times \text{spread}$, a measure of how scattered the comps are (MAD or IQR, both outlier-resistant). Scattered or thin comps pull the value *down on their own*. Every money decision uses that conservative value, never the raw midpoint.

The ceiling, then the call. From the conservative value it strips your target profit and every cost between winning and reselling (eBay fees, buyer-protection fee, postage both ways) by solving for the highest buy price that still clears your profit. That's the **max bid**. If the live price sits under it and the floors clear, it reads

BID; over it **SKIP**; thin/scattered comps **LOW CONF**; no comps **NO DATA**.

What data goes in. Two live feeds plus your watch list. **Comps:** recent sold prices for the exact item (the cards lane already has this; cameras start on a Browse active-listing proxy until real sold data is wired). **Targets:** the live auctions you're watching, pulled ending-soonest. **Your watch list:** the specific cards or camera models to value. Cameras add an identity step first: a messy title ("mk 3", "mark iii") resolves to one exact model, or it's dropped rather than valued on a guess.

Why no machine-learning model? Three reasons, all deliberate. The data is small (a handful of comps per item), so a fitted model would overfit noise. You need to *trust the number* enough to spend real money, and a median you can explain beats a score you can't. And it ships now. If volume grows, a learned ratio-by-condition model is the obvious upgrade, and the calibration log is already collecting the training data for it.

Where it learns: the calibration loop

Every alert is logged with its prediction. When a flip resolves, you fill in what actually happened, and `calibrate.py` reads the gap between predicted and real. It **reports and suggests, never auto-edits**, so you stay in control of every dial. Three things it tells you:

- 1 **Suggested sold ratio**, from real outcomes: $\text{resale} \div \text{comp median}$. This is the main learning signal; it replaces the hand-set 0.85 / 1.0 placeholder once real flips land.
- 2 **Conservative coverage**: the share of flips where resale $\geq$ the conservative value. Want this high; if it's low, the model is over-valuing and k should rise.
- 3 **Profit bias**: median of (real profit – predicted). Positive means it under-promises (safe); negative means it over-promises (tighten the floors).

Run it during the read-only week before trusting any number to spend against: `python calibrate.py`. Cards and cameras share the same loop.

What you can change: every dial, in `config.yaml`

PARAMETER	DOES WHAT	NOW
<code>sold_to_asking_ratio</code>	Sold price as a fraction of comp median. The headline learning dial.	0.85 / 1.0
<code>k</code>	How hard uncertainty is penalised. Higher = more cautious value.	1.0
<code>spread_method</code>	Which outlier-resistant spread to use: <code>mad</code> or <code>iqr</code> .	mad
<code>gate.min_comps / max_rel_dispersion</code>	Confidence gate: stay quiet below N comps or above this scatter.	8 / 0.35
<code>margin_floor / profit_floor</code>	Won't recommend below this % margin or absolute £ profit.	25% / £15
<code>price_cap</code>	Hard ceiling on item price: a risk control, not a ranking input.	£50 / £600
<code>fees.*</code>	Itemised: FVF %, fixed fee, buyer-protection tiers, postage both ways.	all-in
confidence shaping	<code>n_full_confidence</code> , <code>confidence_high/medium</code> set the high/med/low bands.	15 / .66 / .33

Each sector (cards, cameras) overrides only what differs, so one file holds both lanes. Change a value, re-run, see the effect instantly on mock data.

When the eBay keyset lands: the go-live sequence

Everything below is built and tested on mock data; it's gated only on your developer approval, then runs in one pass:

- 1 **Account-deletion notification step:** eBay's one compliance hoop. Read-only, stores no user data (opt out).
- 2 **Wire the keys:** add `EBAY_APP_ID / EBAY_CERT_ID` as repo secrets, switch the live source to `--mode browse`.
- 3 **Self-test a real GB auction:** confirm bid price, bid count and end-date fields come back populated before trusting them. Both lanes.
- 4 **Wire the cameras sold source:** Browse proxy first (ratio $\approx$ 0.85), upgrade to real GBP sold data later (ratio $\rightarrow$ 1.0). One config edit.
- 5 **Read-only week, then calibrate:** let it log predictions against real outcomes, run the loop above, set the ratio and floors from reality, then trust it.

Read-only by design: the bot values and alerts, you place every bid by hand. No autonomous spend, no scraping, no ToS breach. Model lives in `src/valbot/` (valuation, fees, threshold, calibrate); all dials in `config.yaml`. Decisions logged in `docs/adr/`.